

Fluid Simulation Project Report

Eugenio Animalì

1 Introduction

This report describes the steps I took to build a 2D fluid simulation. Starting from Voronoi diagrams, I progressively added weighted (Laguerre) cells, optimal transport to enforce target areas, and finally used these tools to simulate fluid particles under gravity.

2 Voronoi Diagram

The foundation of the project is a Voronoi diagram computed over a set of sites inside the unit square $[0, 1]^2$. For each site P_i , its Voronoi cell is the region of the plane closer to P_i than to any other site. I compute this by starting with the full unit square as a polygon and repeatedly clipping it by the bisector of $[P_i, P_j]$ for every other site P_j .

The clipping follows the Sutherland-Hodgman algorithm: for each edge of the current polygon, I determine whether each vertex is on the correct side of the bisector, and compute the intersection of the edge with the bisector when a crossing occurs. The bisector between P_0 and P_i is the set of points equidistant from both, i.e. satisfying $\|x - P_0\|^2 = \|x - P_i\|^2$.

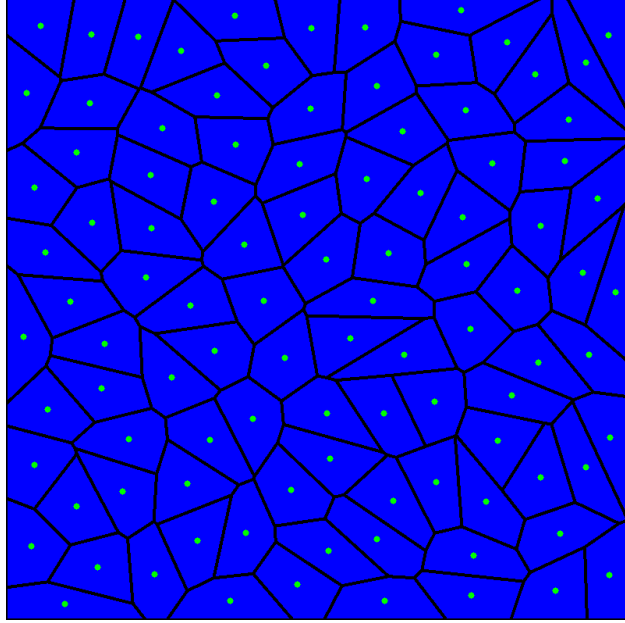


Figure 1: Laguerre Cells

3 Laguerre Cells and Optimal Transport

Apart from some recurring problems with the LBFGS algorithm which persisted throughout the project, implementing the simple Voronoi Diagram, as well as the weighted version with the minimization followed with few problems. The results of the latter are shown in Figure 1, where the weights converged to give cells equal areas of $1/N$.

To control the area of each cell, I moved from standard Voronoi cells to Laguerre (weighted Voronoi) cells. A point x belongs to the Laguerre cell of site P_i with weight w_i if:

$$\|x - P_i\|^2 - w_i \leq \|x - P_j\|^2 - w_j \quad \forall j \neq i.$$

This shifts the bisector between P_i and P_j to a weighted midpoint M' :

$$M' = \frac{P_i + P_j}{2} + \frac{w_i - w_j}{2\|P_i - P_j\|^2}(P_j - P_i).$$

To enforce that each cell has a prescribed target area λ_i , I used semi-discrete optimal transport. The dual objective to maximize is:

$$g(w) = \sum_i \left(\int_{V_i(w)} \|x - P_i\|^2 dx - w_i \cdot |V_i(w)| \right) + \sum_i w_i \lambda_i,$$

which I minimized (as $-g$) using the L-BFGS optimizer. The gradient with respect to w_i is simply $|V_i(w)| - \lambda_i$, the difference between the actual and target area of cell i . The integral $\int_{V_i} \|x - P_i\|^2 dx$ is computed by triangulating each cell from its first vertex and applying the closed-form formula for the integral over a triangle.

4 Partial Optimal Transport for Fluids

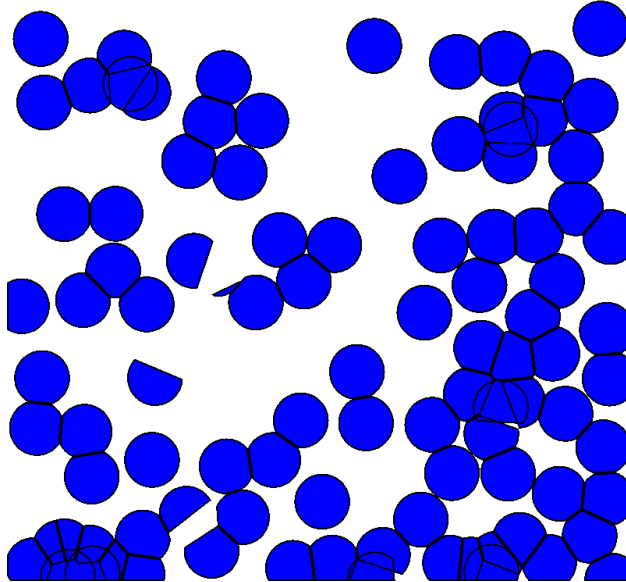


Figure 2: Clipping errors in early circles

In this section, the first issue I faced was in the implementation of clipping circles to reduce the area for each cell. In figure 2, we see issues relative to overlapping circles, as well as some being clipped completely out

of existence. This issue led me to make the incorrect assumption that I should be clipping polygons around their centroid rather than their point of definition, seeing as if a polygon does not include it's original point y_i , and the circle has a radius smaller than the closest distance from the surface of the polygon to that y_i , then the circle would not appear; This was not in fact the problem, and changing this in the implementation gave me issues later on that will be discussed below.

In addition, identifying issues with the mechanics was more difficult due to the intermediary step of the LBFGS algorithm. Any small mistakes in implementation would break the LBFGS optimization, and cause bugs that were hard to identify. The LFBGS function would not give any useful information, (just an unclear -1001 "rounding" error), meaning the gradient descent was not bringing any positive change. Indeed, an example of these issues can be found in the implementation of the Partial Optimal Transport, in which the issue with the incorrect placement of the circles caused the LBFGS to not converge properly as can be seen in Figure 3. In fact, the result doesn't come close to respecting the desired fluid area of 0.4.

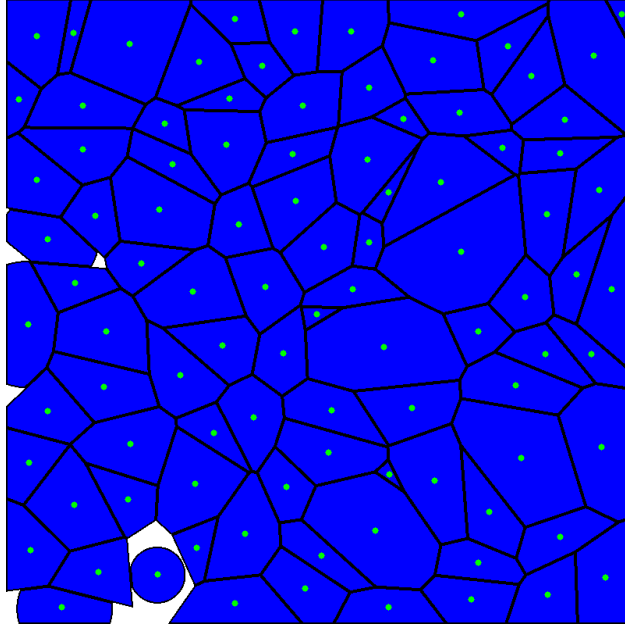


Figure 3: Weights not Converging to Equal Area Cells

To simulate a fluid that does not fill the entire domain, I introduced a partial optimal transport formulation. The N fluid particles are assigned a total area of 0.4 (occupying 40% of the unit square), with the remaining 0.6 treated as "air". This is implemented by adding an extra air site with its own weight w_{air} , and clipping each fluid cell additionally by a disk of radius $r_i = \sqrt{w_i - w_{\text{air}}}$ centered at P_i .

The disk clipping is done by approximating the disk boundary with $M = 80$ line segments and applying the same Sutherland-Hodgman clipping used for bisectors. The gradient with respect to w_{air} is $0.4 - \sum_i |V_i|$, i.e. the error between the target total fluid area and the sum of all fluid cell areas.

5 Fluid Time Step

Each time step of the simulation proceeds as follows. First, I run the optimal transport optimization with the current particle positions to compute the Laguerre cells with the correct target area $\lambda = 0.4/N$ per particle. Then, for each particle i , I compute two forces: a spring force pulling it toward the centroid of its Laguerre cell,

$$F_{\text{spring}} = \frac{c_i - P_i}{\varepsilon^2},$$

and a gravitational force $F_g = m \cdot g$ with $g = (0, -9.81)$. The centroid c_i of the Laguerre cell is computed using the standard polygon centroid formula. I then integrate using symplectic Euler:

$$v_i \leftarrow v_i + \frac{dt}{m}(F_{\text{spring}} + F_g), \quad P_i \leftarrow P_i + dt \cdot v_i.$$

Particles that exit the domain $[0, 1]^2$ are reflected back, reversing the appropriate velocity component.

In the Laguerre section, I had incorrectly understood that the clipping circles were meant to be placed around the centroid of their polygons to avoid disappearing. This caused issues throughout the project, but once the fluid time step was implemented, the issue became apparent, as can be seen in Figure 4. The full video is on the github repository. In this simulation, the 200 cells are placed in a disk above the center of the scene in order to fall and cause a splash. However, the cells do not seem to respect the central disk, as can be seen from some cells lying visibly outside, and being much larger. In fact, the circles are being drawn around the centroids of the Voronoi cells, and given the distribution of points, the cells of the outer points will be very large, reaching the edge of the scene. Thus, the circles are drawn far outside the disk. For the same reason, we can see in the video that when most of the cells fall to the bottom due to gravity, some cells are left floating in the air to fill the voronoi cells of the top of the screen, visibly increasing the pressure on the bottom cells, distorting and squeezing them more. This is what prompted me to change where circles are drawn.

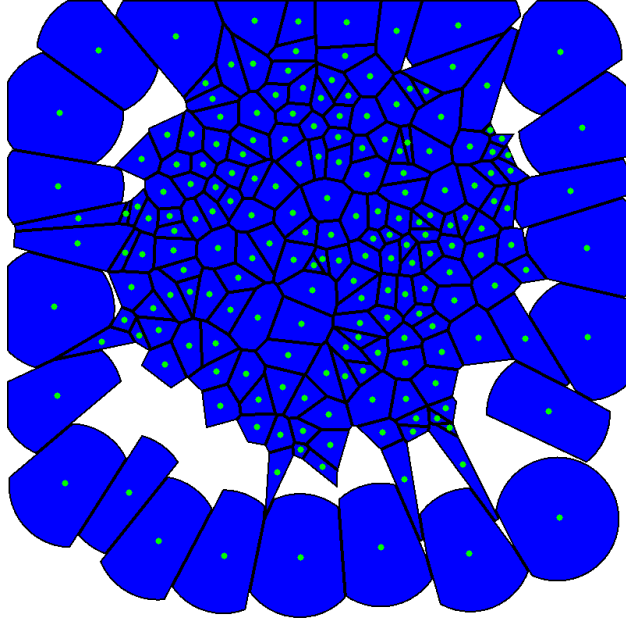


Figure 4: First frame of Centroid-centric Fluid Simulation

6 Final Simulation

Changing this detail solved the issue, and the simulation came out looking much more natural in the end. Using the LBFGS optimization at each frame made frames converge in around 15 seconds each, making a final 600 frame simulation that took 2 hours and a half to generate. The full video is available on the github repository.

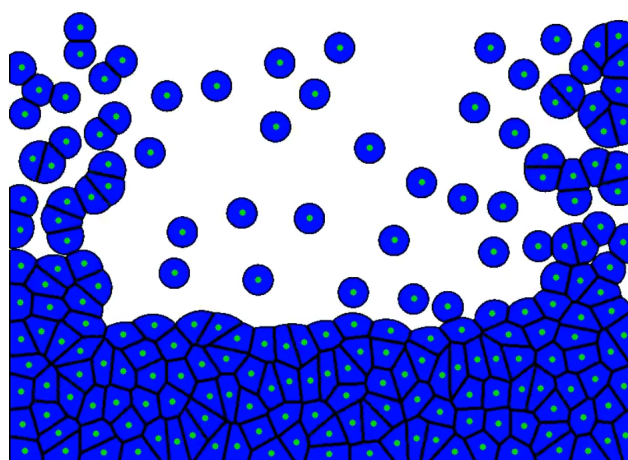


Figure 5: Working Simulation